Autonomous Driving with MacAEV

ELECENG 3EY4: Electrical Systems Integration Project

Instructor: Shahin Sirouspour

Objective: In the final three weeks of the project, we will develop algorithms for self-driving of MacAEV. First, we design a *line-following* controller that enables the vehicle to drive along a hallway by following two side walls. Two lines passing through pairs of LiDAR returns at pre-defined angles on the vehicle left and right sides are used to approximate the side walls. We will then enhance the self-driving algorithm by introducing three new features. We will first explore the *largest gap* in *front* of the vehicle to find the best direction of travel. Two *parallel virtual barrier lines*, one on each side of the best direction of travel, are then found to separate the vehicle from obstacles in its environment. These virtual barrier lines are obtained by formulating and solving a so-called *Quadratic Optimization* with *Linear Inequality* constraints. The objective is to maximize the distance between the virtual barrier lines, while keeping all obstacle points behind them. The line-following controller from the earlier algorithm is used to center the vehicle between the two barrier lines found from solving the optimization problem. Finally, in an optional activity, we will use the onboard RGB-Depth camera data to further enhance the vehicle self-driving functionality by detecting and avoiding obstacles below and above the LiDAR plane in front the of the vehicle.

Safety Instructions:

- Refer to the safety instructions on the course Avenue to Learn website and the course outline before attempting to carry out the activities set out in this document.
- The MacAEV must never powered by the LiPo battery except in the lab and under the supervision of a course TA.
- You must never remove the LiPo batteries from the lab rooms.
- You must only use the wall-mounted switching power supply to power the MacAEV for activities that can be performed outside the regular laboratory hours at home.
- When driving the MacAEV, you must ensure that it is always within your reach so you could safely stop it in case of an emergency.
- The vehicle would sound a beeping alarm if its battery were running too low. In such case, you should immediately stop operating the vehicle and switch the power off. Recharge the depleted the battery.

General Remarks:

- In this document, the generic heading term “Activity” is used for actionable items as well as questions. These items must be specifically addressed in your written report.

- The symbol denotes an Activity that can be performed outside the laboratory at home prior to attending the lab, or after the lab if the activity requires answering questions based on your observations in the lab. You are strongly encouraged to complete these activities outside the laboratory hours to make efficient use of your lab time.
- The symbol denotes an Activity that must be carried out in the lab under TA supervision by powering the MacAEV from the LiPO battery.

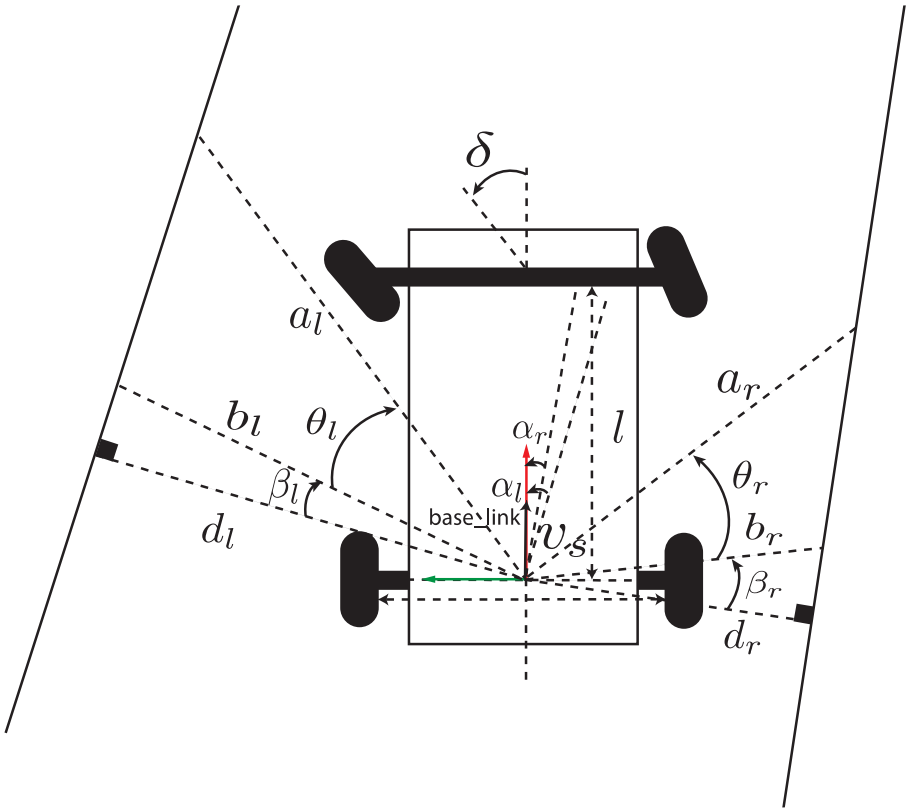

Figure 1: Schematic of the line-following algorithm.

1.1 Self-Driving by Following Side Walls

A self-driving vehicle should be able to follow road lanes. To this end, the vehicle needs to detect the lane by processing on-board sensors information such as data from a camera video feed. This would require the use of image processing techniques such as segmentation and line detection. More recently, Deep Neural Networks (DNN) have been utilized for image segmentation to detect road lanes and other features of the road environment at pixel level (see here for example). The vehicle would use a feedback control loop to stay in the lane by continuously adjusting its steering angle and speed in response to lane tracking errors and presence of other road users. In this part of the project, we will develop a self-driving algorithm based on a similar principle but using the LiDAR scan data instead. In our context, a hallway replaces the road, and the LiDAR data substitutes the camera data. We will utilize the LiDAR scan data to detect the corridor walls. From the vehicle control perspective, what we do here is somewhat similar to vison-based lane tracking control. In our approach, we will estimate the vehicle distance from the side walls using the LiDAR scan. We will develop a feedback controller to maintain the vehicle in the middle of the two walls by continuously adjusting the vehicle steering angle in response to wall distance tracking errors. The controller combines a so-called *feedback-linearization* component with a *Proportional-Derivative (PD)* controller. We will also continuously adjust the vehicle forward velocity in response to any obstacle in front of it.

1.2 Line-Following Control Algorithm

Consider the case where the vehicle is moving in a corridor along two side walls. For the vehicle to self-drive safely in such scenario, it would have to maintain a safe distance from the walls while moving forward with a prescribed velocity. In this case, it would make more sense to control the vehicle distance from the walls than to control its absolute x and y positions with respect to a fixed frame. This approach would have the advantage of not requiring the vehicle absolute position coordinates with respect to a fixed odometry frame. Hence, the controller can be implemented without localizing the vehicle. Referring to the schematics in Fig. 1, let d_r and d_l denote the vehicle distances to the right and left walls, respectively. Moreover, the difference between the left and right distances is defined as $d_{lr} = d_l - d_r$. Then, one can write:

$$\dot{d}_r = v_s \sin \alpha_r \tag{1}$$

$$\dot{d}_l = -v_s \sin \alpha_l \tag{2}$$

$$\dot{d}_{lr} = \dot{d}_l - \dot{d}_r = -v_s \sin \alpha_l - v_s \sin \alpha_r \tag{3}$$

By taking the derivative of these equations with respect to time,

$$\ddot{d}_r = v_s\, \dot{\theta} \cos \alpha_r = \frac{v_s^2}{l} \cos \alpha_r \tan \delta \tag{4}$$

$$\ddot{d}_l = -v_s \dot{\theta} \cos \alpha_l = -\frac{v_s^2}{l} \cos \alpha_l \tan \delta \tag{5}$$

$$\ddot{d}_{lr} = -v_s\, \dot{\theta}(\cos \alpha_l + \cos \alpha_r) = \frac{-v_s^2}{l}(\cos \alpha_l + \cos \alpha_r) \tan \delta \tag{6}$$

In the above derivations, we have used the fact that $\dot{\alpha}_r = \dot{\alpha}_l = \dot{\theta}$. These equations represent *second-order nonlinear* dynamics for the distances of the vehicle to the right wall, left wall, and difference between the distances to the left and right walls. A sensible strategy would be to control the difference between the distances to left and right walls. We define the following control tracking error:

$$\tilde{d}_{lr} = d_{lr} - d_{lr}^{des} \tag{7}$$

Here, d_{lr}^{des} is the desired difference between the left and right distances, normally set to *zero,* so the vehicle can settle right in the middle of the hallway. The steering angle δ is used as the primary input for the line-following control. We choose the steering angle command as:

$$\delta = \operatorname{atan}\left(\frac{-l}{v_s^2(\cos \alpha_r + \cos \alpha_l)}\left(-k_p\, \tilde{d}_{lr} - k_d\, \dot{d}_{lr}\right)\right) \tag{8}$$

By substituting the control law (8) into the system second-order dynamics in (6), this *feedback-linearizing* control law renders the closed-loop dynamics of the system to those of a standard second-order linear time invariant (LTI) system in the form of:

$$\ddot{d}_{lr} + k_d\, \dot{d}_{lr} + k_p\, d_{lr} = k_p d_{lr}^{des} \tag{9}$$

The transfer function of the closed-loop control system can be obtained by applying the Laplace transform to both sides of the above differential equation, and is given by:

$$H_{cl}(s) = \frac{D_{lr}(s)}{D_{lr}^{des}(s)} = \frac{k_p}{s^2 + k_d s + k_p} \tag{10}$$

Remark: In practice, you must constrain the commanded steering angle within its physical limit, $\delta_{\max}$, i.e.,

$$\delta_c = \begin{cases} \delta_{max} & \text{if } \delta > \delta_{max} \\ \delta & \text{if } -\delta_{max} \le \delta \le \delta_{max} \\ -\delta_{max} & \text{if } \delta < -\delta_{max} \end{cases} \tag{11}$$

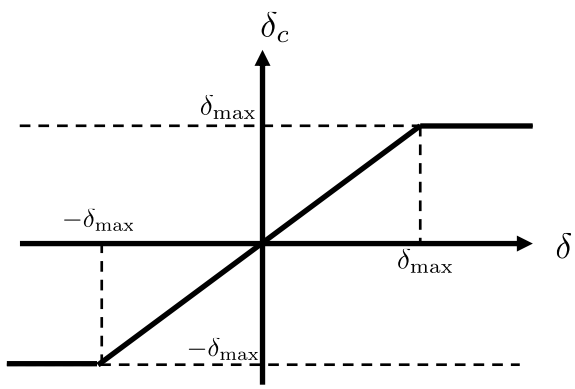

Figure 2: Constrained steering command

Activity 1:

Determine the *steady-state* tracking error in response to a *constant* command in the closed-loop control. Explain how, in theory, the parameters k_d, k_p would affect the closed-loop control system response.

The above feedback controller combines *feedback linearization* with a PD controller. Feedback linearization is a popular approach for controlling nonlinear dynamical systems. The controller implementation requires the angles α_l, α_r and the distances to the walls d_l, d_r. Referring to Fig.1,

$$\beta_r = \operatorname{atan}\left(\frac{a_r \cos\theta_r - b_r}{a_r \sin\theta_r}\right) \tag{12}$$

$$\beta_l = \operatorname{atan}\left(\frac{a_l \cos\theta_l - b_l}{a_l \sin\theta_l}\right) \tag{13}$$

with

$$\theta_r = \angle a_r - \angle b_r \tag{14}$$

$$\theta_l = \angle b_l - \angle a_l \tag{15}$$

The angles α_l, α_r can be computed as:

$$\alpha_r = \beta_r + \frac{\pi}{2} - \angle b_r \tag{16}$$

$$\alpha_l = -\beta_l + \frac{3\pi}{2} - \angle b_l \tag{17}$$

Note that the "zero" angle reference for the LiDAR scan angles is the x axis of the "laser" frame which is in the opposite direction of the x axis of the "base_link" frame. The distances to the right and left walls are given by:

$$d_r = b_r \cos\beta_r \tag{18}$$

$$d_l = b_l \cos\beta_l \tag{19}$$

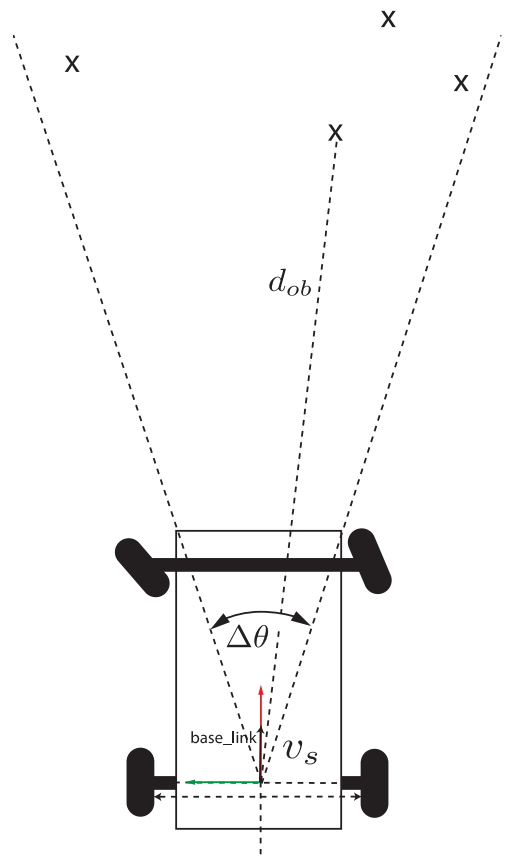

Figure 3: The closest obstacle point in front of vehicle within a user-defined narrow viewing angle $\Delta\theta$.

1.3 Vehicle Velocity Control

The Line-following controller uses only one of the control inputs, i.e., steering angle δ, to control the distances of the vehicle to the left and right walls. Ideally, we would like to drive the vehicle with a nominal velocity v_s^d, while maintaining the desired distances to the walls. The vehicle velocity can be adjusted based on its distance to obstacles in front of it for safer driving. To achieve this objective, we look for the closest obstacle point in a narrow viewing angle in the front of the vehicle (see Fig. 3) and choose the vehicle velocity v_s using the following expression:

$$v_s^c = v_s^d\left[1 - \exp^{-\max(d_{ob}-d_{stop},0)/d_\tau}\right] \tag{20}$$

Here v_s^d is a user-set nominal driving velocity, v_s^c is the actual velocity command sent to the vehicle, d_{ob} is the distance to closest obstacle in the viewing angle $\Delta\theta$ in front of the vehicle, d_{stop} is a safe stop distance, and d_τ is a design parameter that controls the rate of decay in the exponential.

Activity 2:

Explain what velocity control strategy in (19) tries to accomplish.

The Line-following Algorithm is summarized in Table 1.

Table 1: The Line-following Self-Driving Control Algorithm
Inputs: Current LiDAR scan data z_k and vehicle velocity from VESC, v_s **Outputs:** Vehicle steering angle δ_c and velocity v_s^c commands • Calculate the right and left walls parameters from (12)-(17) (see Fig. 1) • Calculate the distances to the right and left walls from (18)-(19) • Calculate the steering angle command δ_c from (8) and (11) (set $\delta_{\max} = 0.41$ rad for MacAEV) • Find the distance to closest obstacle point in a user-defined narrow viewing angle in front of the vehicle (d_{ob} in Fig. 3) and calculate the vehicle velocity command v_s^c from (20)

1.4 Controller Implementation

Activity 3:

Copy the map files "env_map.pgm" and "env_map.yaml" in "/maps" directory of F1THENTH_SIMULATOR package and make sure that the "simulator.launch" is modified to use this map in your simulations. You should replace the following files in your existing software stack with the ones provided on Avenue to Learn for Lab 7:

behavior_conrtoller.cpp
mux.cpp
simulator.launch
experiment.launch

This would ensure that the *behavior_controller* and *mux* nodes are set up properly to respond to the Joystick command for selecting self-driving mode. Once you have replaced these files, you should rebuild the simulator package by moving to the root directory of the catkin workspace containing the simulator package and running the following command:

```
$ catkin_make
```

You should also add the new control algorithm parameters to the file "params.yaml" so you can adjust their values in this file, as opposed to doing it so directly in the code.

You have been provided with the template file "navigation_linefollowing_template.py" on Avenue as a starting point for developing a new navigation node in Python. You may alternatively

develop a C++ node from the template file "navigation_linefollowing_template.cpp", if you prefer programing in C++.

Your navigation node must accomplish the following objectives:

- Read the controller parameters from the "params.yaml" file.
- Subscribe to the LiDAR scan and Wheel Odometry topics so you can read the LiDAR scan data and the actual vehicle velocity as they become available.
- Create a publisher to a drive topic so you can send the computed velocity and steering angle commands to the VESC driver.
- Read vehicle velocity v_s in the odometry callback function.
- Read the LiDAR scan data in the LiDAR callback function; compute the steering angle δ_c to position vehicle in the middle of the left and right-side walls. Also, compute the vehicle drive velocity command v_s^c.
- Publish the steering and velocity commands using AckermannDriveStamped message format with the topic name "/drive" so they can be sent over to the VESC.

You must include your complete navigation node source code with your lab report submission.

1.5 Self-driving with Line-Following Algorithm in Simulation

Activity 4:

The following values are suggested as a good starting point for various control parameters. You are encouraged to adjust these values to fine tune the response of the vehicle based on your simulations, and later, your experiments.

$$\angle b_l = 270°, \angle a_l = 220°, \angle b_r = 90°, \angle a_r = 140°$$
$$k_p = 6,\ k_d = 4$$
$$v_s^d = 2.0m/s,\ \Delta\theta = 11.25°, d_{stop} = 0.6m, d_\tau = 1.0m$$
$$\delta_{\max} = 0.41$$

Modify your "params.yaml" to include these new controller parameters in the file. Launch the Simulator. The self-driving mode can be activated/de-activated by toggling the "RB" on Joystick. While in the self-driving model, pressing the "LB" should immediately switch over to the manual driving mode. Observe the vehicle behavior in the simulation environment as it drives along the hallway. Where do you see the algorithm struggling and why? Explore and explain the impact of the controller gains k_p and k_d on the response characteristics. How would the mechanical rotation frequency of the LiDAR impact the selection of these controller gains?

1.6 Self-Driving MacAEV with Line-following Algorithm

Activity 5:

Make sure the environment around the vehicle is free of people and safe for operating the vehicle. Launch "experiment.launch" to start the vehicle. Remember to always keep the vehicle within safe reach so you would not lose the communication between the Joystick and the vehicle on-board computer.

1) Do you observe any difference in the vehicle behavior between the simulations and the experiment? How do you explain these potential differences?
2) The RPLiDAR A2M8 scanner has two important parameters, the number of samples/second and mechanical scan frequency. You can refer to the RLiDAR specifications here to find out about the values of these parameters and their range. Discuss the trade-offs involved affecting the selection of these parameters in the context of self-driving. For example, with a fixed signal sampling rate, what happens if you increase or decrease mechanical rotation frequency? What is the impact of increasing or decreasing sampling rate with a fixed mechanical rotation frequency?

2 Self-Driving with Virtual Separating Barriers

In the previous part, we developed and implemented a self-driving algorithm by following two walls on the sides of the vehicle. This strategy usually performs well where there are clearly defined straight walls, e.g., inside a hallway, and with no obstacle in front of the vehicle. However, it can struggle in absence of such features, around sharp turns, or when obstacles appear in front of the vehicle. Recall that the distance to each wall was determined using only two pre-defined LiDAR scan beams. In environments with non-flat walls or generally without clearly defined walls, this could lead to erratic self-driving behaviour due to rapid changes in the orientation and/or distance of the estimated wall with respect to the vehicle as the points used for wall estimation could quickly change.

We will now develop a more robust way of defining "walls", or virtual separating barriers, for the vehicle to follow. To help avoid colliding with obstacles, first the *largest obstacle-free gap* in front of the vehicle is identified. The vehicle direction of travel is set along the direction of the furthest point in this gap. Next, two *parallel* virtual line barriers are constructed one on each side of the direction of travel using *multiple* LiDAR returns. This new approach for constructing these virtual line barriers significantly improves on the previous method that used only two pairs of points based on the assumption of straight walls. The virtual barrier lines provide the vehicle with largest safe gap to travel along, with all the obstacle points sitting behind them. This new way of selecting "walls" to follow should markedly enhance the performance of the self-driving algorithm in the

presence of obstacles, around corners, and when the walls are not straight or have dents or protrusions.

2.1 Best Direction of Travel

The new algorithm needs to find the *best direction of travel* for the vehicle first. The basic idea is that at every time step the vehicle should try to avoid the nearest obstacles within a *safety zone* around it. The algorithm in Table 2 accomplishes this objective.

Table 2: Algorithm for Finding Best Direction of Travel

Inputs: current LiDAR scan data z_k
Output: best direction of travel

- Define a Field of View (FOV) in front of the vehicle. This is where we will look for potential obstacles to avoid (e.g., $\pm 70°$ with respect to the x-axis of "base_link" in Fig. 1)
- Set to *zero* the range values for all LiDAR returns that fall within a safe distance in FOV in front of the vehicle. The zero range values represent immediate obstacles within the safety zone and non-zero LiDAR returns are considered *free space*.
- Find the *longest* sequence of *consecutive* non-zero LiDAR returns among the *free-space* points; this represents the largest gap in front of the vehicle.
- Find the best direction of travel for the vehicle in the largest gap in front of it. Let (θ_i, r_i) denote a pair of LiDAR beam angles and the corresponding range measurements. One possible choice for the desired heading direction would be towards the furthest point in the largest gap, i.e.,

$$\theta_{des} = \theta_{i_M} \text{ and } i_M = \arg\max r_i \tag{21}$$

Alternatively, each LiDAR beam angle can be weighted by the range of its return, and the best direction of travel can be set to the weighted average of the LiDAR beam angles, i.e.,

$$\theta_{des} = \frac{\sum_i r_i \theta_i}{\sum_i r_i} \tag{22}$$

The summation in (22) is computed over all the LiDAR beams in the largest gap.

2.2 Virtual Separating Line Barriers

Fig. 4 illustrates a hypothetical scenario with the vehicle and the LiDAR returns from one complete scan (only LiDARs within FOV are considered). The LiDAR returns from obstacles are

denoted by × symbols. Recall that in the previous line-following algorithm we used the two pairs of LiDAR beams (b_l, a_l) and (b_r, a_r) to find two lines on the left and right sides of the vehicle. Here, the virtual line barriers on the left and right sides of the vehicle are computed based on a user-defined number of LiDAR returns that fall within the corresponding LiDAR beams (b_l, a_l) and (b_r, a_r), defined with respect to the best direction of travel θ_{des}. The algorithm for computing θ_{des} was given in Table 2. The green x and red x symbols denote the obstacle points that need to be placed behind the virtual barriers. This new approach should yield a more robust and realistic rendition of the line barriers between the vehicle and obstacles particularly in environments without clearly defined straight walls. We would also want the pair of parallel lines to have the *greatest* possible distance from each other. This is to provide the vehicle with the largest possible obstacle-free corridor to drive through.

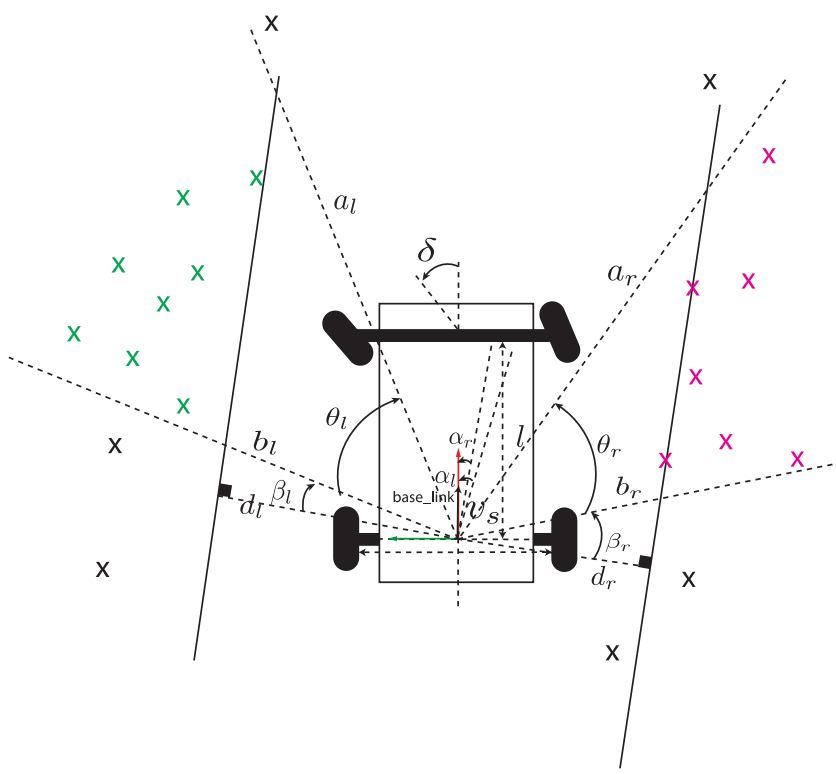

Figure 4: Schematics for virtual barrier-based self-driving control algorithm; left and right virtual separating barrier lines must be parallel and have the greatest possible distance from each other; note that the two sectors θ_r and θ_l are defined with respect to the best direction of travel

In the next section, we will explore how to find the separating line barriers by formulating and solving a so-called *Inequality Constrained Quadratic Program* (IQP). Refer to the lecture notes for information about IQP and how it can be solved numerically.

2.3 Formulation of Quadratic Program

The goal here is to find a pair of parallel lines with the *greatest* distance from each other that would separate the vehicles from select LiDAR returns that fall between two pre-defined pair of beam angles, i.e., see Fig. 4. The idea is inspired by the concept of Support Vector Machines (SVM) (see here or here for reference). The two parallel lines in Fig. 4 are parameterized as follows:

$$w^T p + s = 1 \tag{23}$$
$$w^T p + s = -1 \tag{24}$$

where $w \in \mathbb{R}^2$ is a vector of two parameters, $s \in \mathbb{R}$ is a scaler; these together constitute the three unknown line parameters to be found by solving an optimization problem. The vector $p \in \mathbb{R}^2$ represents the coordinates of any point on the line with respect to the "base_link" frame. Note that the vector w is perpendicular to the two parallel lines. We must ensure that the origin of the "base_link" frame with coordinates $p = [0 \quad 0]^T$ is between the two parallel lines. This requires that:

$$-1 + \epsilon \leq s \leq 1 - \epsilon \tag{25}$$

Here $\epsilon > 0$ is a small number. The gap, i.e., the distance between the two lines, is given by:

$$d = d_r + d_l = \frac{2}{\sqrt{w^T w}} \tag{26}$$

To maximize the gap size in (26), one can simply minimize $w^T w$. Let us denote the red x and green x obstacle points by the indices i and j, respectively. Then,

$$w^T p_i + s \geq 1 \tag{27}$$
$$w^T p_j + s \leq -1 \tag{28}$$

These inequalities ensure that the obstacle points are properly separated from the vehicle by the parallel virtual line barriers. The optimization problem for finding the parallel separating line barriers with the largest gap is formulated as an IQP:

$$\begin{aligned} \min_{w,s} \quad & \frac{1}{2} w^T w \\ & s.t. \\ w^T p_i + s - 1 \geq 0 \quad & \forall i \\ w^T p_j + s + 1 \leq 0 \quad & \forall j \\ -1 + \epsilon \leq s \leq 1 - \epsilon & \end{aligned} \tag{29}$$

2.4 Modified Line-Following Control Algorithm

We must now modify the wall-following algorithm to incorporate the new method for computing the separating line barriers between the vehicle and obstacles. Referring to Fig. 1 & Fig. 4, the distances to the left and right barriers are given by:

$$d_l = \frac{1}{\sqrt{w_l^T w_l}}, \qquad d_r = \frac{1}{\sqrt{w_r^T w_r}} \tag{30}$$

where w_l and w_r are computed from w and b, the solution to the optimization problem in (28) as follows:

$$w_r = \frac{w}{s-1} \tag{31}$$

$$w_l = \frac{w}{s+1} \tag{32}$$

Next, we need to compute the rate of change in the distances to the barriers. Before that, let us define the normalized (unit) vectors:

$$\widehat{w}_l = \frac{w_l}{\|w_l\|} = d_l w_l, \qquad \widehat{w}_r = \frac{w_r}{\|w_r\|} = d_r w_r \tag{33}$$

Using these definitions, it can be shown that:

$$\dot{d}_l = [v_s \quad 0]\widehat{w}_l \tag{34}$$

$$\dot{d}_r = [v_s \quad 0]\widehat{w}_r \tag{35}$$

Moreover,

$$\cos\alpha_l = [0 \quad -1]\widehat{w}_l \tag{36}$$

$$\cos\alpha_r = [0 \quad 1]\widehat{w}_r \tag{37}$$

At this point, we should be able to use the wall-following controller to regulate the difference between the distances to the left and right separating barrier lines. The algorithm for self-driving with virtual separating barriers is summarized in Table 3.

Table 3: Algorithm for Self-Driving with Parallel Virtual Separating Barriers
Inputs: current LiDAR scan data z_k and vehicle velocity from VESC, v_s **Outputs:** vehicle steering δ_c and velocity command v_s^c • Using the latest LiDAR scan data, z_k, compute the Best Direction of Travel based on the Algorithm given in Table 2. • Solve the QP in (29) to find w, s, the parameters of two parallel lines that separate the vehicle from obstacles on left and right side along the Best Direction of Travel • Compute the variables needed in feedback linearizing+PD controller from (30)-(37) • Compute the steering angle command δ_c from (8) and (11) • Find the distance to closest obstacle point in a user-defined narrow viewing angle in front of the vehicle (d_{ob} in Fig. 3) and calculate the vehicle velocity command v_s^c from (20)

2.5 Quadratic Program (QP) Solver

You can use an open-source implementation of the Active Constraints Set method of Goldfarb and Idnani[1] to solve the Quadratic Program (QP) in (29).

Python node: For a Python-based node, you can install and use *quadprog*. Execute the following command to install *quadprog* in your Python environment:

```
$ sudo pip install quadprog
```

The *quadprog* solver assumes an QP of the following form:

$$\begin{aligned} \min_{x} \quad & \frac{1}{2} x^T G x - a^T x \\ & s.t. \\ & C^T x - b \geq 0 \end{aligned} \tag{38}$$

Here, $x \in \mathbb{R}^n$ is the vector of optimization decision variables, $a \in \mathbb{R}^n$, $C \in \mathbb{R}^{n \times m}$, $b \in \mathbb{R}^m$ are constant matrices and vectors.

The QP solver can be called using the following function call:

x = solve_qp(G, a, C, b, meq)[0]

[1] D. Goldfarb and A. Idnani (1983). A numerically stable dual method for solving strictly convex quadratic programs. Mathematical Programming, 27, 1-33.

where G, a, C, b are defined in the problem formulation in (38), and "meq" is the number of strict equality constraints (*meq* =0 in our problem). Note that this formulation of the QP is slightly different from the one presented in the lecture notes, which the formulation in (29) is based on. Aside from the differences in notations, here the equality and inequality constraints are combined, with the variable "meq" denoting the number of equality constraints. For further information about Quadratic Programing in Python, and specifically *quadprog* see the link here.

The optimization formulation in (29) can be stated as the one in (38) with the following variable definitions:

$$G = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 0.0001 \end{bmatrix}_{3\times 3}$$

$$a = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}_{3\times 1}$$

$$C = \begin{bmatrix} p_1^{xr} & p_2^{xr} & \cdots & p_{n_r}^{xr} & -p_1^{xl} & -p_2^{xl} & \cdots & -p_{n_l}^{xl} & 0 & 0 \\ p_2^{yr} & p_2^{yr} & \cdots & p_{n_r}^{yr} & -p_2^{yl} & -p_2^{yl} & \cdots & -p_{n_r}^{yl} & 0 & 0 \\ 1 & 1 & \cdots & 1 & -1 & -1 & \cdots & -1 & 1 & -1 \end{bmatrix}_{3\times(n_r+n_l+2)}$$

$$b = \begin{bmatrix} 1 \\ 1 \\ \vdots \\ 1 \\ -0.99 \\ -0.99 \end{bmatrix}_{(n_r+n_l+2)\times 1}$$

Here $p_i = \begin{bmatrix} p_i^{xr} \\ p_i^{yr} \end{bmatrix}$ and $p_j = \begin{bmatrix} p_j^{xl} \\ p_j^{yl} \end{bmatrix}$ are the coordinates of the obstacle points on the right (red) and left (green) sides of the vehicle with respect to the *base_link* frame. Moreover, n_r and n_l are the number of obstacle points on the right and left sides of the vehicle, respectively.

Remark: Note that a small positive value has been assigned to $G(3,3)$. This would translate into an additional term $0.5 \times 10^{-4} s^2$ to $0.5\, w^T w$ in the cost function of the optimization problem in (29). While this added cost term is negligible and would not alter the ultimate solution in any meaningful way, it would ensure that matrix $G > 0$ is positive definite. This is a requirement of the *quadprog* solver.

C++ node: If you choose to develop a C++ node, you should install and use *QuadProg++*, which also implements the Goldfarb and Idnani's active constraints set method. *QuadProg++* assumes the optimization problem is formulated as:

$$\begin{aligned} &\min_{x} \frac{1}{2} x^T G x + g_0^T x \\ &\quad s.t. \\ &C_E^T x + c_{e0} = 0 \\ &C_I^T x + c_{i0} \geq 0 \end{aligned} \tag{39}$$

Here $x \in \mathbb{R}^n$ is the vector of optimization decision variables, $g_0 \in \mathbb{R}^n$, $C_E \in \mathbb{R}^{n \times p}$, $C_I \in \mathbb{R}^{n \times m}$, $c_{e0} \in \mathbb{R}^m$, $c_{i0} \in \mathbb{R}^m$ are constant matrices and vectors. Note that unlike in *quadprog* formulation in (38), the strictly equality constraints are separated from the rest of the constraints in (39). Refer to here for more details about how to use this solver.

Activity 6:

Implement the self-driving control algorithm in Table 3 within a new Python or C++ navigation node. You are provided with incomplete Python template file as a starting point for this activity. Your new node should accomplish the following tasks:

- Read the algorithm parameters from the "params.yaml" file.
- Subscribe to the LiDAR scan and Wheel Odometry topics so you can read the LiDAR scan data and the vehicle velocity v_s.
- Create a publisher to a drive topic so you can send the velocity and steering angle commands to the VESC.
- Read vehicle velocity v_s in Odometry callback function.
- Read LiDAR scan data in LiDAR callback function.
- Find the best direction of travel.
- Set up and solve the QP problem to find two parallel barrier lines.
- Compute the steering angle δ_c to position vehicle in the middle of the left and right-side barrier lines.
- Compute the vehicle velocity command v_s^c.
- Publish the steering and velocity commands using AckermannDriveStamped message format with the topic name "/drive" so they can be sent over to the VESC.

You must include the completed navigation node source code along with your lab report submission.

2.6 Self-Driving with Virtual Separating Barriers in Simulation

Activity 7:

Copy the map files "hallway_w_blocks.pgm" and "hallway_w_blocks.yaml" in "/maps" directory of F1THENTH_SIMULATOR package and make sure that the "simulator.launch" is modified to use this map in your simulations. Use the simulator environment to fine-tune the values of the control algorithm parameters until you achieve a satisfactory response. Based on your observations, what are the key parameters of the algorithm that affect its performance in your simulations? What are the trade-offs in selecting the values of these parameters? Demonstrate to your TA that you have successfully accomplished this task.

2.7 Self-Driving with Virtual Separating Barriers in Experiment with MacAEV

Activity 8:

Make sure the environment around the vehicle is free of people and safe for vehicle operation and launch the file "experiment.launch". Recall that the self-driving mode can be activated and de-activated at by toggling the "RB" on Joystick. Note that pressing the "LB" would activate the manual driving mode. You can use either of these options to stop self-driving in case of an emergency. Remember to always keep the vehicle in your reach so you would not lose the connection between the vehicle and the Joystick. Adjust the controller parameters as you see necessary to achieve a satisfactory response. Briefly report on your observations from the experiments.

3 Enhancing Self-driving Functionality by Augmenting Data from RGB-D Camera

Activity 9: (Bonus, 5% of the total course grade)

The MacAEV's 2D LiDAR can miss obstacles that are outside its scanning plane. For example, the LiDAR is very likely unable to detect another MacAEV in the environment because of its mounting position on the vehicle. Your task in this optional activity is to modify the self-driving algorithm in Section 2 of this document so it can take advantage of the RGB-D camera depth image information for detecting and avoiding obstacles undetectable by the LiDAR. You should develop a new modified navigation node that accomplishes the following additional objectives:

- Subscribes and listens to the Depth Image data and Depth Information topics published by the Realsense RGB-Depth camera.
- Obtains the depth camera intrinsic parameters from the data published on the Depth camera information topic.

- Retrieves the depth image data from the Depth Image topic.
- Converts the depth image pixels to points in 3D space using the camera model and camera intrinsic parameters ((see lecture notes on Depth from Stereo Vision)
- From camera-reported obstacle points in 3D space, finds those that can potentially collide with the vehicle but are undetected by the LiDAR. You should be careful to exclude ground points since they can never collide with the vehicle. These are unsafe obstacle points discovered by the RGB-Depth camera.
- Augments the LiDAR scan data by projecting unsafe obstacle points from the previous step onto the LiDAR plane.
- Executes the self-driving algorithm with the augmented LiDAR scan data.

Note that processing the Depth image data can be computationally involved. You should attempt to reduce these computations, when possible, for example by sub-sampling the depth image rows and columns as you see appropriate. You could also consider fitting a plane to the depth image points that have possibly arisen from reflections from the ground. This fitted plane could then serve as an effective tool for identifying whether a point from the depth image in 3D space is from the ground or for an obstacle. Here is an example of algorithm for fitting plane a group of points in 3D space with associated code.